$F2 \parallel WC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing the maximum weighted completion time in a two-machine flow shop, denoted $F2 \parallel WC_{\max}$, models lines on which the most costly order paces the shop. We establish that the problem is strongly NP-hard and give a mixed integer programming formulation for it. We then show that an optimal schedule preserves the permutation property and has a block structure of one block per distinct weight, Johnson's order within each block, and the blocks in non-increasing weight order. On this structure we build three methods. An exact dynamic program prunes dominated states and runs in $O(nKV^{3K-3})$ time, where K is the number of distinct weights and V is the total processing time. A 2-approximation runs in $O(n \log n)$ time, and rounding the time coordinates and weights of the dynamic program turns it into a fully polynomial-time approximation scheme that attains a $(1+\varepsilon)$-approximation for a constant number of weights. For large instances we develop three heuristics, an adapted NEH heuristic, an ant colony optimization (ACO) algorithm (Zhang et al., 2023) and a genetic algorithm (Reeves, 1995). The computational study compares the exact methods on small instances and the heuristics on the large ones, measured against a theoretical lower bound. On the small instances the dynamic program returned the same values as the model and was faster by up to two orders of magnitude. Among the heuristics the genetic algorithm returned the smallest gap in every setting, ahead of the NEH heuristic and of the ant colony optimization.

Keywords: Flow shop; Dynamic programming; Approximation algorithms;

1 Introduction

The two-machine flow shop, $F2$ in the standard notation, is one of the best known models in scheduling. Its structure is that of a plant whose jobs are machined on one machine and

*Corresponding author. E-mail address: author@email.com

finished on another. Metal parts are cut and then coated, and a casting is rolled after it is poured. A web is printed and then bound, and a granulate is compressed after it is mixed. A job must leave the first stage before it can enter the second, and each stage carries one job at a time. Johnson settled the sequencing decision for this model as early as 1954 (Johnson, 1954). The two-stage structure has been a standard subject of sequencing theory ever since, and its variants and solution methods are surveyed in (Pinedo, 2022). The criterion of this paper is the maximum weighted completion time, also called the weighted makespan. Written $WC_{\max}$, it charges each job the product of its weight and its completion time and takes the largest of those products. Where the makespan asks when the shop falls idle, the weighted makespan asks which job is left waiting and what that wait costs. The weight belongs to the order rather than to the plant as a whole, and in a plant it is rarely a continuum. A contractual penalty for lateness, the capital tied up in work in process and the priority a large customer has bought are three classes rather than a spectrum. The most costly order is the one carrying the largest of them.

Much the same structure has appeared outside the factory: a system that serves large language model agents has two such stages. A step is reasoned about in the chain of thought (Wei et al., 2022), and acted upon, by calling a tool or running code, in the chain of execution (Yao et al., 2023). Reasoning and acting are interleaved in practice, but they occupy distinct resources. The order in which the steps are admitted to them settles their completion times.

Serving a task takes many such steps, such as a retrieved document, a tool call or a subquery. The order in which the steps are admitted settles when the task as a whole finishes. The steps are not equal: one bought by a paid tier, or one whose failure is costly, weighs more than the rest. With the reasoning and the acting as the two machines and the steps as the jobs, the two-machine flow shop follows. Each stage carries one job at a time, and what decides whether the service is acceptable is the completion time of the most costly step. That completion time is weighted by the cost of the step, so the criterion is the maximum weighted completion time rather than the average.

That criterion, on that machine environment, is what this paper studies under the name $F2 \parallel WC_{\max}$. It matters for the systems above, where a wrong sequence delays the most costly step rather than the average one. It matters for scheduling theory as well, where a maximum weighted criterion has not been placed in the two-machine flow shop.

In scheduling theory the criterion has been studied in several machine environments. On a single machine, jobs are revealed over time and must be sequenced without knowledge of the future. For this online setting, best-possible algorithms are known for the maximum weighted completion time (Li, 2015). A single machine carrying one restricted rate-modifying activity has also been treated for the same criterion (Zuo et al., 2026). On identical parallel machines, randomized approximation schemes for the weighted makespan have been developed (Sun, 2024). The criterion has also been combined with the decision of which orders to accept (Lu et al., 2025). Maximum-type criteria have been studied in other forms as well. For a

single machine, the maximum earliness/tardiness cost has been minimized when jobs may be rejected (Guo et al., 2026). In all of these settings, however, a job occupies one machine for its entire processing time. The interaction between two consecutive stages, which is what defines the flow shop, therefore does not arise.

The flow shop environment, by contrast, has been studied extensively but for different criteria. The variant that has attracted most theoretical attention is the proportionate flow shop, in which every machine spends the same time on a given job. That model, the ordered flow shop and related variants are surveyed in (Panwalkar et al., 2013). Its appeal is that it can often be reduced to a single-machine problem with the same criterion (Panwalkar and Koulamas, 2015), so that results proved for one machine carry over. Weighted objectives have been treated within it as well, but the objective treated is the sum of the weighted completion times rather than their maximum (Shakhlevich et al., 1998). For the general two-machine flow shop, the criterion that has been studied is the makespan, most recently together with the decision of which orders to reject (Shabtay and Gerstl, 2024).

The criterion and the machine environment are therefore each well established. Their combination is not, and the works closest to this paper show how far apart the two remain. In the criterion group, every setting reviewed above keeps each job on a single machine for its whole processing time. This holds for the online single-machine results (Li, 2015), the rate-modifying variant (Zuo et al., 2026), the parallel-machine approximation schemes (Sun, 2024) and the order-acceptance model (Lu et al., 2025). In the environment group, the weighted objective that has been treated is a sum rather than a maximum (Shakhlevich et al., 1998). For the general two-machine flow shop, the criterion treated is the makespan (Shabtay and Gerstl, 2024).

The problem is not solved by combining these results, for two reasons. The reductions that make the proportionate flow shop tractable require the processing times to be proportionate. The general two-machine flow shop does not satisfy that assumption, so results proved for one machine do not carry over. More fundamentally, a total weighted completion time decomposes over the jobs. It can therefore be accumulated while a schedule is built, which is what allows a dynamic program to compare partial schedules with one another. A maximum does not decompose in this way, since a partial schedule carries no information about its final value until it is complete. Optimizing the maximum weighted completion time in a two-machine flow shop therefore requires structural properties of a kind that the earlier work does not supply.

In this paper we study the maximum weighted completion time in a two-machine flow shop, denoted $F2 \parallel WC_{\max}$. Four features distinguish it from the settings reviewed above. (i) Two machines stand in series, so that each completion time depends on the idle time inherited from its predecessors. (ii) The objective is a maximum over weighted completion times rather than their sum. (iii) The processing times are general, with no proportionate assumption. (iv) No job may be rejected, so that every job must be scheduled. Replacing the

weighted sum by a weighted maximum, and dropping the proportionality assumption, would already put the problem beyond the single-machine reduction. We show that $F2 \parallel WC_{\max}$ is in fact strongly NP-hard (Theorem 3.2). The difficulty is structural rather than a matter of the instance size. Because the objective is determined by a single job, an exchange that shortens many completion times while lengthening that one does not improve the objective value. Because the jobs carry unequal weights, moreover, Johnson's rule has no direct extension to the weighted case.

Our results follow from the structure of an optimal schedule. Since the objective is a maximum, jobs of equal weight are interchangeable with respect to it. An optimal schedule can therefore be assumed to have a block structure, with one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. On this structure we build three methods, an exact dynamic program, a 2-approximation and an approximation scheme. The dynamic program runs in $O(nKV^{3K-3})$ time, where K is the number of distinct weights and V is the total processing time. It is practical when the weights fall into few classes, as in the plants described above. A 2-approximation runs in $O(n \log n)$ time, and rounding the input of the dynamic program gives a fully polynomial-time approximation scheme. The scheme attains a $(1+\varepsilon)$-approximation for constant K. Because the exact methods remain exponential in K, we also develop three heuristics for large instances and report a computational study that compares them with the 2-approximation.

The rest of the paper is organized as follows. Section 2 fixes the notation and states the completion-time recurrence. Section 3 gives the mixed integer programming formulation and establishes the complexity of the problem. Section 4 proves the structural properties, develops the exact dynamic program, and presents the 2-approximation and the approximation scheme. Section 5 describes the three heuristics. Section 6 reports the computational study, and Section 7 concludes.

2 Preliminaries

We consider the problem $F2 \parallel WC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series. Each job J_j must be processed first on M_1 for a_j time units and then on M_2 for b_j time units, and preemption is not allowed. All processing times a_j and b_j are assumed to be positive integers. Each machine can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing sequence on both machines. For a schedule π, let $A_j = \sum_{h=1}^{j} a_{\pi(h)}$ and $B_j = \sum_{h=1}^{j} b_{\pi(h)}$ be the cumulative processing times on M_1 and M_2 after the first j jobs. The completion time of the job in position j on machine M_2 is

$$C_{\pi(j)} = \max_{1 \le i \le j} \Big\{ \sum_{h=1}^{i} a_{\pi(h)} + \sum_{h=i}^{j} b_{\pi(h)} \Big\}. \tag{1}$$

The objective is to find a schedule π that minimizes $WC_{\max}$. Throughout, $WC^*_{\max}$ denotes the optimal value of $F2 \parallel WC_{\max}$.

The problem $F2 \parallel WC_{\max}$ can be formulated as a mixed integer linear program (MILP). For a permutation schedule, the positions are indexed by $k = 1, \ldots, n$. We introduce the following decision variables:

- $x_{jk} = 1$ if job J_j is assigned to position k, and 0 otherwise;
- C^1_k completion time of the job in position k on M_1;
- C^2_k completion time of the job in position k on M_2;
- C_j completion time of job J_j on M_2;
- Z the maximum weighted completion time $WC_{\max}$;
- M a sufficiently large constant, taken as $2\sum_{j=1}^{n}(a_j + b_j)$.

The MILP model is:

$$\min \quad Z \tag{2}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, \ldots, n \tag{3}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, \ldots, n \tag{4}$$

$$C^1_1 = \sum_{j=1}^{n} a_j x_{j1} \tag{5}$$

$$C^1_k = C^1_{k-1} + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, \ldots, n \tag{6}$$

$$C^2_1 = C^1_1 + \sum_{j=1}^{n} b_j x_{j1} \tag{7}$$

$$C^2_k \geq C^1_k + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{8}$$

$$C^2_k \geq C^2_{k-1} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{9}$$

$$C_j \geq C^2_k - M(1 - x_{jk}), \quad j = 1, \ldots, n, \; k = 1, \ldots, n \tag{10}$$

$$Z \geq w_j C_j, \quad j = 1, \ldots, n \tag{11}$$

$$x_{jk} \in \{0, 1\}, \tag{12}$$

$$C_k^1, C_k^2, C_j \geq 0 \tag{13}$$

Constraint (3)–(4) ensure a one-to-one assignment between jobs and positions. Constraints (5)–(6) compute the completion time of the job in each position on M_1. Constraints (7)–(9) compute the completion time of the job in each position on M_2. For $k \geq 2$, the job in position k starts on M_2 only after it has completed M_1 and the preceding job has released M_2. Constraint (10) links the position-based completion times to the job-based completion times through a big-M formulation: when $x_{jk} = 1$, job J_j is in position k and $C_j \geq C_k^2$; when $x_{jk} = 0$, the right-hand side becomes $C_k^2 - M \leq 0$ and the constraint is inactive. Constraint (11) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$. Constraint (12) requires the assignment variables x_{jk} to be binary. Constraint (13) requires the completion times C_k^1, C_k^2 and C_j to be nonnegative.

3 Problem formulation

Theorem 3.1. *$F2 \parallel WC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given r positive integers $x_1, \ldots, x_r$ with $\sum_{i=1}^{r} x_i = 2T$, do there exist disjoint subsets $Z_1, Z_2 \subseteq \{1, \ldots, r\}$ with $Z_1 \cup Z_2 = \{1, \ldots, r\}$ and

$$\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T?$$

Construct an instance of $F2 \parallel WC_{\max}$ with $n = r + 1$ jobs. For $j = 1, \ldots, n-1$, set $a_j = 1$, $b_j = r x_j$ and $w_j = T + 1$; for the remaining job J_n, set $a_n = rT$, $b_n = 1$ and $w_n = 2T + 1$. The decision question is whether some schedule satisfies

$$WC_{\max} \leq Y, \qquad \text{where} \qquad Y = r(T+1)(2T+1).$$

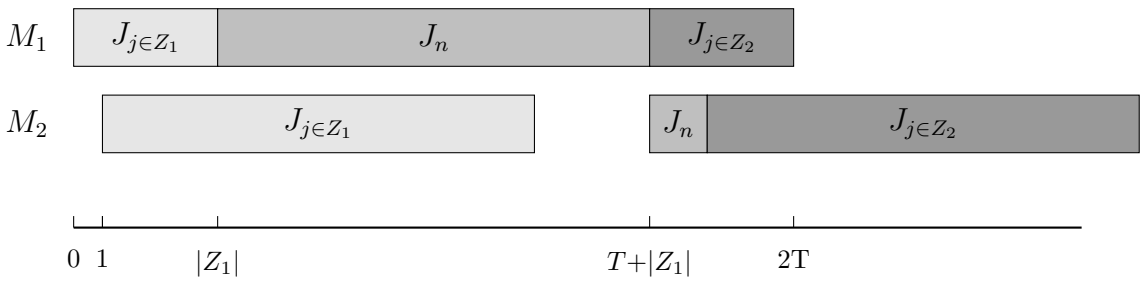


Figure 1

The jobs $J_1, \ldots, J_{n-1}$ are partitioned into two subsets by J_n; the resulting layout on the two machines is shown in Figure 1. The total M_2 processing times of the two subsets are

unknown, but their sum equals $2rT$. We may therefore write them as $r(T+\xi)$ and $r(T-\xi)$ for some $-T \le \xi \le T$. The completion times of J_n are then

$$C_n^1 = |Z_1| + rT + 1, \tag{14}$$

$$C_n^2 = 1 + r(T+\xi) + 1, \tag{15}$$

and $C_n = \max\{C_n^1, C_n^2\}$.

If $\xi = 0$, then $\sum_{j\in Z_1} b_j = \sum_{j\in Z_2} b_j$, so the Partition instance has a feasible solution. In this case C_n is given by (14), and

$$\begin{aligned} C_n &= |Z_1| + rT + 1 \le rT + r = r(T+1), \\ w_n C_n &= (2T+1)\, r(T+1) = Y, \\ WC_{\max} &= r(T+1)(C_n + rT) = Y, \end{aligned}$$

so $w_j C_j \le WC_{\max} = Y$.

Conversely, suppose that some schedule satisfies $WC_{\max} \le Y$. When $\xi > 0$, C_n is given by (15), and

$$\begin{aligned} C_n &= 1 + r(T+\xi) + 1 \ge 1 + r(T+1) + 1, \\ w_n C_n &\ge (2T+1)\big(1 + r(T+1) + 1\big) = Y. \end{aligned}$$

When $\xi < 0$, C_n is given by (14), and

$$\begin{aligned} C_{\max} &= C_n + \sum_{j\in Z_2} b_j \\ &= 1 + rT + |Z_1| + r(T-\xi) \\ &\ge rT + r + rT \\ &= r(2T+1) = Y, \\ WC_{\max} &\ge r(T+1)\big(r(2T+1)\big) = Y, \end{aligned}$$

so $WC_{\max} > Y$, a contradiction.

The Partition instance therefore admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \le Y$. □

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the strongly NP-complete 3-Partition problem: given $3m$ positive integers $x_1, \ldots, x_{3m}$ and an integer B satisfying $B/4 < x_i < B/2$ and $\sum_{i=1}^{3m} x_i = mB$, can the integers be partitioned into m triples, each of sum B?

Let $R = 3m + 5$, $D = RB + 4$, and $T = mD$. We construct an instance with $3m$ element jobs $J_1, \ldots, J_{3m}$ and m separator jobs $S_1, \ldots, S_m$:

- for every element job J_j, set $a_j = 1$, $b_j = Rx_j + 1$;
- for every separator job S_k, set $a_{S_k} = RB + 1$, $b_{S_k} = 1$.

All processing times are positive integers. For $k = 1, \ldots, m$, define $d_k = kD + 1$ and let $Q = (T + 2)^3$. Set

$$w_{S_k} = \left\lfloor \frac{Q}{d_k} \right\rfloor, \qquad w_j = \left\lfloor \frac{Q}{T} \right\rfloor \quad (j = 1, \ldots, 3m),$$

and use the decision threshold $Y = Q$. Since $d_k \le T + 1$ and $Q \ge d_k(d_k + 1)$, we have $\lfloor Q/d_k \rfloor > d_k$. Thus, for every $d \in \{d_1, \ldots, d_m, T\}$ and every integer completion time C,

$$\left\lfloor \frac{Q}{d} \right\rfloor C \le Q \quad \Longrightarrow \quad C \le d. \tag{16}$$

Indeed, if $C \ge d + 1$, write $Q = qd + r$ with $q = \lfloor Q/d \rfloor$ and $0 \le r < d$. Then $q(d + 1) = Q - r + q > Q$ because $q > d > r$. Suppose first that the 3-Partition instance is a yes-instance. Let $G_1, \ldots, G_m$ be triples with $\sum_{J_j \in G_k} x_j = B$ for every k, and schedule

$$G_1, S_1, G_2, S_2, \ldots, G_m, S_m.$$

The three element jobs in G_k use 3 units on M_1, and S_k uses $RB + 1$ units. Hence S_k completes on M_1 at time kD. The three element jobs in G_k use $RB + 3$ units on M_2; since the preceding separator completes on M_2 at time $(k - 1)D + 1$, the last element of G_k completes at time kD. Therefore

$$C_{S_k} = kD + 1 = d_k.$$

Every element job completes on M_2 no later than $T = mD$. By the definition of the weights, all weighted completion times are at most Q, so $WC_{\max} \le Y$.

Conversely, suppose that a schedule π satisfies $WC_{\max} \le Y$. Since the objective is regular, the standard permutation-schedule property for two-machine flow shops lets us restrict attention to permutation schedules. The separator jobs have identical processing times, so if their positions are fixed, exchanging the labels of two separators that occur in the wrong order can only decrease the maximum weighted completion time, because $w_{S_1} \ge w_{S_2} \ge \cdots \ge w_{S_m}$. We may consequently assume that the separators occur in the order $S_1, S_2, \ldots, S_m$. By (16),

$$C_{S_k} \le kD + 1 \quad (k = 1, \ldots, m), \qquad C_j \le T \quad (j = 1, \ldots, 3m).$$

No element job can occur after S_m. Indeed, if at least one element job did so, consider the last such element job. By the permutation-schedule property invoked above, the processing sequences on M_1 and M_2 coincide, so this job is also the last job on M_1. All m separators

and all $3m$ element jobs therefore complete on M_1 no later than it, so its M_1 completion time is at least the total M_1 processing time

$$m(RB+1)+3m = m(RB+4) = T.$$

Its M_2 completion time would then be strictly greater than T, contradicting $C_j \le T$. The element jobs are therefore partitioned into the m groups G_k lying between S_{k-1} and S_k, where S_0 denotes the start of the schedule. Put

$$B_k = \sum_{J_j \in G_k} x_j, \qquad E_k = \sum_{h=1}^{k} |G_h|, \qquad E_0 = 0.$$

For $k = 2, \dots, m$, the M_2 path gives

$$C_{S_k} \ge C_{S_{k-1}} + RB_k + |G_k| + 1, \tag{17}$$

while the M_1 path to S_{k-1} gives

$$C_{S_{k-1}} \ge (k-1)(RB+1) + E_{k-1} + 1. \tag{18}$$

Combining (17) and (18) with $C_{S_k} \le kD + 1$ yields

$$R(B_k - B) + E_k \le 3k, \qquad k = 2, \dots, m. \tag{19}$$

For $k = 1$, the M_2 path and $C_{S_1} \le D + 1$ similarly give

$$R(B_1 - B) + E_1 \le 4. \tag{20}$$

Because $R = 3m + 5$, (19) and (20) imply $B_k \le B$ for every k. Otherwise $B_k \ge B + 1$, and the respective left-hand side exceeds $3k$, or exceeds 4 when $k = 1$. Since every element job belongs to one of the groups, $\sum_{k=1}^{m} B_k = mB$, and hence $B_k = B$ for every k. Finally, $B/4 < x_j < B/2$ implies that a group with sum B contains exactly three elements: at most two elements have sum strictly less than B, while at least four elements have sum strictly greater than B. Thus $G_1, \dots, G_m$ is a valid 3-partition.

The construction uses $4m$ jobs, and all processing times, weights and the threshold are bounded by a polynomial in m and B. Since 3-Partition remains NP-complete when B is polynomially bounded in m, this is a strong polynomial reduction, and $F2 \parallel WC_{\max}$ is strongly NP-hard. □

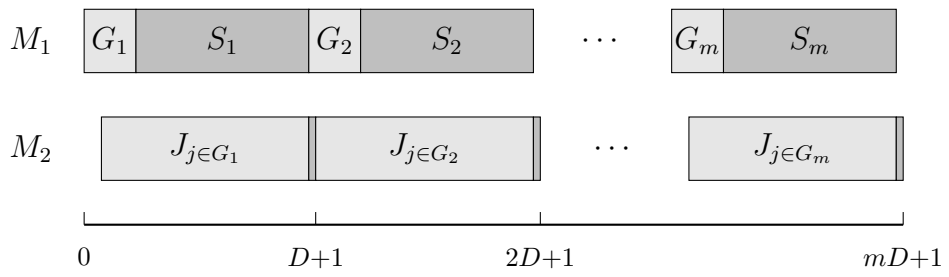


Figure 2: 3-Partition reduction

4 Algorithm

In this section, we design a dynamic programming algorithm for $F2 \parallel WC_{\max}$ and analyze its running time. Throughout, let K denote the number of distinct weights and $V = \sum_{j=1}^{n}(a_j + b_j)$ the total processing time.

Since the objective is a maximum over weighted completion times, jobs of equal weight are interchangeable. An optimal schedule can therefore be assumed to have a block structure: one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. A schedule is thus fixed once we decide how each weight class is distributed among the blocks. This is what makes a dynamic program possible, and why the running time of every algorithm below is governed by K.

We build three algorithms on this structure. The first algorithm is exact, and it prunes dominated states. A state records each block's processing time on both machines, the completion time of its last job on M_2, and that job's weight. It runs in $O(n \cdot K \cdot |\mathscr{H}|_{\max})$ time, with $|\mathscr{H}|_{\max} \leq 2^K V^{3K-3}$. The second sorts the jobs by non-increasing weight in $O(n \log n)$ time and returns a schedule within a factor 2 of the optimum. The third rounds the time coordinates and the weights to geometrically spaced grids, which turns the dynamic program into a $(1+\epsilon)$-approximation scheme for a fixed number of distinct weights.

4.1 The number of weights is constant

For $F2 \parallel WC_{\max}$, an optimal schedule exists in which both machines process the jobs in the same order (Baker, 1974); we therefore restrict attention to such permutation schedules. Within this class, Johnson's rule (Johnson, 1954) minimizes the makespan: the jobs with $a_j \leq b_j$ are scheduled first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

Fix a permutation schedule π. Let $W_1 > W_2 > \cdots > W_K$ be the K distinct weights occurring among the jobs, and let

$$\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}, \qquad k = 1, \ldots, K,$$

denote the corresponding weight classes. Set $\ell_0 = 0$. For $k = 1, \ldots, K$ in sequence, let ℓ_k be the position of the last job of weight W_k among the jobs not yet assigned to a block, that is, among $\pi(\ell_{k-1}+1), \ldots, \pi(n)$; if no such job remains, set $\ell_k = \ell_{k-1}$. The blocks are the consecutive segments

$$\mathcal{B}_k = \{\pi(\ell_{k-1}+1), \ldots, \pi(\ell_k)\}, \qquad k = 1, \ldots, K,$$

so that in particular $\mathcal{B}_1 = \{\pi(1), \ldots, \pi(\ell_1)\}$. Equivalently, once the blocks $\mathcal{B}_1, \ldots, \mathcal{B}_k$ have been removed, the remaining jobs are

$$\mathcal{J} \setminus \bigcup_{i=1}^{k} \mathcal{B}_i = \{\pi(\ell_k+1), \ldots, \pi(n)\}, \qquad k = 1, \ldots, K.$$

Hence $\ell_1 \le \ell_2 \le \cdots \le \ell_K$, and no block extends beyond its own last job. For each block $\mathcal{B}_i$, let A_i and B_i denote the total processing time of its jobs on M_1 and M_2, respectively, C_i the completion time of its last job on M_2, and L_i the weight of its last job. If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job; such a block is called empty and satisfies $A_k = B_k = C_k = L_k = 0$, and it may host any job of weight at most W_k. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight W_k, whereas an empty block has $L_k = 0$. Since $W_1 > W_2 > \cdots > W_K$, the block-end weights are non-increasing on the non-empty blocks.

We illustrate the block construction with a small example.

Example 4.1. *Consider a six-job instance with sequence $\pi = (J_1, \ldots, J_6)$ and the following (a_j, b_j, w_j) values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The weight classes are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, and $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$, and the resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, and $\mathcal{B}_3 = \{J_6\}$.

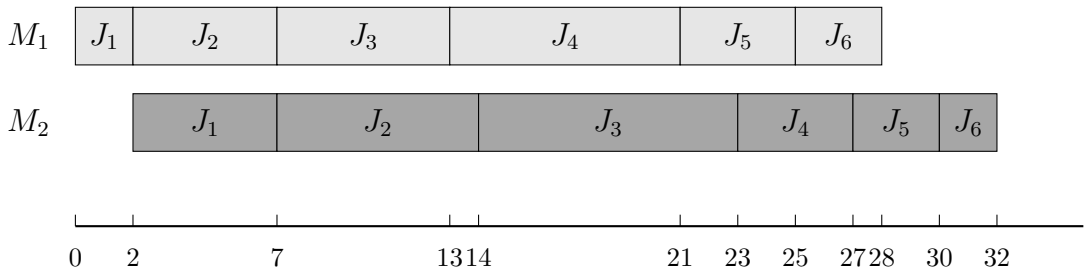


Figure 3

Lemma 4.1. *For any schedule π, rescheduling the jobs within each block according to Johnson's rule (Johnson, 1954) and concatenating the blocks in their original order yields a schedule π' with $WC_{\max}(\pi') \le WC_{\max}(\pi)$.*

Proof. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight L_k, and every job in $\mathcal{B}_k$ has weight at most L_k: a weight occurring in $\mathcal{B}_k$ either equals W_k, or it has already appeared in an earlier block, whose weights are larger. Let $C_k(\sigma)$ denote the completion time of the last job of $\mathcal{B}_k$ under schedule σ. An empty block satisfies $L_k C_k(\sigma) = 0$ and therefore contributes nothing to the objective, so the argument below applies to the non-empty blocks.

We first show that the maximum weighted completion time within a block is attained by the last job of that block. For any $J_j \in \mathcal{B}_k$, the last job of the block completes no earlier than J_j, so $C_j(\sigma) \le C_k(\sigma)$, while $w_j \le L_k$ by the block construction. Hence $w_j C_j(\sigma) \le L_k C_k(\sigma)$,

and taking the maximum over $j \in \mathcal{B}_k$ gives

$$\max_{J_j \in \mathcal{B}_k} w_j C_j(\sigma) = L_k C_k(\sigma).$$

The objective therefore equals

$$WC_{\max}(\sigma) = \max_{k=1,\dots,K} L_k C_k(\sigma),$$

so it depends only on the last job of each block.

Let π' be the schedule obtained by rescheduling the jobs within each block $\mathcal{B}_k$ according to Johnson's rule, keeping the blocks in their original order. We prove $C_k(\pi') \leq C_k(\pi)$ for every k by induction on k.

Base case $k = 1$. Block $\mathcal{B}_1$ starts at time zero on both machines regardless of its internal order, so Johnson's rule gives $C_1(\pi') \leq C_1(\pi)$.

Inductive step. Assume $C_{k-1}(\pi') \leq C_{k-1}(\pi)$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \dots, \mathcal{B}_{k-1}$, which is invariant under internal rescheduling, so $\mathcal{B}_k$ begins at the same instant on M_1 under both schedules. On M_2, block $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^{k} A_i,\, C_{k-1}\}$; the first term is identical under π and π', and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than under π. Together with the same M_1 start and the internally optimal Johnson scheduling, this gives $C_k(\pi') \leq C_k(\pi)$, which completes the induction.

Finally,

$$WC_{\max}(\pi') = \max_k L_k C_k(\pi') \leq \max_k L_k C_k(\pi) = WC_{\max}(\pi),$$

which establishes the lemma. □

4.2 Algorithm Properties and Dynamic Programming

We now develop a dynamic programming algorithm that processes the jobs sequentially in the Johnson sequence and assigns each job to one of the K blocks $\mathcal{B}_1, \dots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis for the dynamic program. Given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in the order of non-increasing block weights yields a schedule whose $WC_{\max}$ value is no larger than that of any other schedule respecting the same assignment. For each $j = 1, \dots, n$, let $\mathscr{H}_j$ denote the set of states attainable after the first j jobs have been processed. A state records the following quantities for every block $\mathcal{B}_i$:

- A_i: the total processing time on machine M_1 of the jobs in block $\mathcal{B}_i$;
- B_i: the total processing time on machine M_2 of the jobs in block $\mathcal{B}_i$;
- C_i: the completion time of the last job of block $\mathcal{B}_i$ on machine M_2;

- L_i: the weight of the last job of block $\mathcal{B}_i$.

Since $W_1 > \cdots > W_K$, the blocks eligible for J_j form a prefix. Writing

$$i^* = \max\{\, i : w_j \le W_i \,\},$$

the eligible set is $\mathcal{E}(J_j) = \{\, 1, \ldots, i^* \,\}$. Each state is therefore the aggregate K-block tuple

$$(A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ C_1, \ldots, C_K;\ L_1, \ldots, L_K),$$

which is described by $4K$ parameters.

The initial state set contains a single state whose components are all zero: $\mathscr{H}_0 = \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$. In Algorithm 1, T_{A_i} denotes the total M_1 processing time of the first i blocks, and T_{B_i} the completion time of block $\mathcal{B}_i$ on M_2.

Lemma 4.2. *Let $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$ and $\mathscr{H}' = (A'_1, \ldots, A'_K; B'_1, \ldots, B'_K; C'_1, \ldots, C'_K; L'_1, \ldots, L'_K)$ be two states in $\mathscr{H}_j$. If $A_i = A'_i$, $B_i = B'_i$ and $L_i = L'_i$ for every i, and $C_i \le C'_i$ for every i, then $\mathscr{H}'$ can be discarded without affecting the optimal solution.*

Proof. Consider any continuation that extends $\mathscr{H}'$ to a complete schedule Π'. Replacing the prefix represented by $\mathscr{H}'$ with the prefix represented by $\mathscr{H}$ yields a schedule Π that assigns the remaining jobs identically. For every block $\mathcal{B}_k$ that follows the updated one, the M_2 start time is $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, and C_{k-1} is no larger under $\mathscr{H}$ than under $\mathscr{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathscr{H}$. Since A_i, B_i and L_i are identical in the two states, the objective value $WC_{\max}$ of Π does not exceed that of Π'. □

For $j = 1, \ldots, n$, the algorithm extends $\mathscr{H}_{j-1}$ to $\mathscr{H}_j$ by placing J_j at the end of each eligible block. That is, for every state in $\mathscr{H}_{j-1}$ and every $i \in \mathcal{E}(J_j)$, only the i-th slice of the state is updated while all other blocks remain unchanged:

$$A'_i = A_i + a_j, \quad B'_i = B_i + b_j, \quad C'_i = \max\{C_i + b_j,\ A_i + a_j + b_j\}. \tag{21}$$

The resulting candidate states are collected in temporary sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$. By Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathscr{H}_j$.

To support Step 4, each state in $\mathscr{H}_j$ stores a pointer to the state in $\mathscr{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. Starting from a state in $\mathscr{H}_n$ attaining Z^*, backtracking along these pointers recovers the block $\mathcal{B}_i$ containing each job J_j. Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Theorem 4.1. *Algorithm 2 solves $F2 \parallel WC_{\max}$ optimally in $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ time, where $|\mathscr{H}|_{\max} \le 2^K V^{3K-3}$ and $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. For constant K, this is $O(nV^{3K-3})$.*

Algorithm 1: Block concatenation and evaluation

Input: State $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$.

Step 1. [Initialization] Set $T_{A_0} \leftarrow 0$, $T_{B_0} \leftarrow 0$, $WC_{\max} \leftarrow 0$.
Step 2. [Evaluation] **for** $i = 1$ **to** K **do**
  $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
  $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
  if $L_i > 0$ **then**
    $WC_{\max} \leftarrow \max\{\, WC_{\max},\ L_i \cdot T_{B_i} \,\}$
  end
end

Output: $WC_{\max}$.

Algorithm 2: DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; weights $W_1 > \cdots > W_K$.

Step 1. [Initialization] Set $L_i \leftarrow 0$ for $i = 1, \ldots, K$, set
  $\mathscr{H}_0 \leftarrow \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$ and $Z^* \leftarrow +\infty$.
Step 2. [Generation] **for** $j = 1$ **to** n **do**
  for *each* $(A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_{j-1}$ **do**
    for *each block* $i \in \mathcal{E}(J_j) = \{1, \ldots, i^*\}$ **do**
      $A'_i \leftarrow A_i + a_j$, $B'_i \leftarrow B_i + b_j$, $L'_i \leftarrow w_j$
      $C'_i \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
      Add the state
       $(A_1, \ldots, A'_i, \ldots, A_K; B_1, \ldots, B'_i, \ldots, B_K; C_1, \ldots, C'_i, \ldots, C_K; L_1, \ldots, L'_i, \ldots, L_K)$
       to $\mathscr{H}_j$
    end
  end
end
[Elimination] Elimination $\mathscr{H}_j$ for $j \in \{1, \ldots, n\}$ by Lemma 4.2
Step 3. [Calculate] **for** *each state*
  $S = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_n$ **do**
  if $L_i \in \{0, W_i\}$ *for every* $i = 1, \ldots, K$ **then**
    $WC_{\max} \leftarrow$ Algorithm 1 applied to S
    $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$
  else
    Discard S.
  end
end
Step 4. [Result] Recover a schedule attaining Z^* by standard backtracking.

Output: An optimal schedule σ.

Proof. Each state in $\mathscr{H}_j$ is the $4K$-tuple

$$(A_1, \ldots, A_K;\, B_1, \ldots, B_K;\, C_1, \ldots, C_K;\, L_1, \ldots, L_K),$$

so it suffices to bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of the jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0,\, V]$. Since the j jobs considered so far form a prefix of the fixed Johnson sequence, $\sum_{i=1}^{K} A_i$ equals the sum of the a_h over that prefix. At most $K-1$ of the A_i are therefore independent, contributing at most $V^{\,K-1}$ distinct A-vectors. Symmetrically, $\sum_{i=1}^{K} B_i$ is the prefix sum of the b_j, so the B_i components likewise contribute at most $V^{\,K-1}$ distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when that block is scheduled in isolation by Johnson's rule. Since $C_i \le A_i + B_i \le V$, the C-vector contributes at most V^{K-1} distinct values. The weight L_i equals W_i when block $\mathcal{B}_i$ is non-empty and 0 when it is empty, so the L-vector takes at most 2^K distinct values. Multiplying these bounds gives

$$|\mathscr{H}_j| \le 2^K \cdot V^{\,K-1} \cdot V^{\,2K-2} = 2^K \cdot V^{\,3K-3}.$$

In iteration j, the algorithm takes each state in $\mathscr{H}_{j-1}$, tries every block $i \in \{1, \ldots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into $\mathcal{Y}_i$. The merge step collects the K sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$ and removes duplicates in time proportional to the total number of generated tuples. Iteration j therefore costs $O(K \cdot |\mathscr{H}_{j-1}|)$, and summing over the n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation by Algorithm 1 evaluates each state in $\mathscr{H}_n$ in $O(K)$ time, which is dominated by the DP phase. For fixed K, 2^K is constant and the bound becomes $O(n \cdot V^{\,3K-3})$. □

4.3 A 2-Approximation Algorithm

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.4 has running time exponential in the number K of distinct weights. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Theorem 4.2. *Algorithm 3 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \le 2\, WC^{*}_{\max}$.*

Algorithm 3: 2-Approximation for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

1 **Step 1.** Sort the jobs by non-increasing weight w_j (ties broken arbitrarily).

2 **Step 2.** Schedule the jobs in this sequence on both machines.

Output: The resulting permutation schedule σ.

Proof. Let σ be the schedule returned by Algorithm 3. Its jobs appear in non-increasing order of weight, so we name them $J_1, J_2, \ldots, J_n$, with $w_1 \geq w_2 \geq \cdots \geq w_n$ and $\sigma = (J_1, J_2, \ldots, J_n)$. Every job of the prefix $S = \{J_1, J_2, \ldots, J_j\}$ then carries a weight of at least w_j.

We first bound the completion times in σ. By (1), the job J_j completes on M_2 at time $C_j(\sigma)$, and every term of that maximum is bounded by the full sum:

$$\sum_{h=1}^{i} a_h + \sum_{h=i}^{j} b_h \leq \sum_{h=1}^{j} a_h + \sum_{h=1}^{j} b_h = \sum_{h \in S} \big(a_h + b_h\big), \qquad 1 \leq i \leq j.$$

Hence

$$C_j(\sigma) \;\leq\; \sum_{h \in S} \big(a_h + b_h\big). \tag{22}$$

Since $WC_{\max}(\sigma) = \max_{1 \leq j \leq n} w_j\, C_j(\sigma)$, multiplying (22) by $w_j > 0$ and taking the maximum over j gives

$$WC_{\max}(\sigma) = \max_{1 \leq j \leq n} w_j\, C_j(\sigma) \leq \max_{1 \leq j \leq n} w_j \sum_{h \in S} \big(a_h + b_h\big). \tag{23}$$

It remains to bound the right-hand side by $2WC^*_{\max}$. Fix a prefix $S = \{J_1, J_2, \ldots, J_j\}$ and let π^* be an optimal schedule. Let J_u be the job of S that finishes last on M_1 in π^*, and J_v the job of S that finishes last on M_2. All jobs of S are processed on M_1 before J_u completes M_1, so $C^*_u \geq \sum_{h \in S} a_h$. In the same way, all M_2 processing of the jobs of S precedes the completion of J_v on M_2, so $C^*_v \geq \sum_{h \in S} b_h$. Since both J_u and J_v lie in S, their weights satisfy $w_u, w_v \geq w_j$. The value $WC^*_{\max}$ bounds the weighted completion time of every job of π^*, so $WC^*_{\max} \geq w_u C^*_u \geq w_j \sum_{h \in S} a_h$ and $WC^*_{\max} \geq w_v C^*_v \geq w_j \sum_{h \in S} b_h$.

The prefix S was arbitrary, so the largest of these quantities over the prefixes bounds the optimum as well, and a single job bounds it from below on its own. Three lower bounds on the optimum follow,

$$LB_1 = \max_{1 \leq j \leq n} w_j \sum_{h \in S} a_h, \tag{24}$$

$$LB_2 = \max_{1 \leq j \leq n} w_j \sum_{h \in S} b_h, \tag{25}$$

$$LB_3 = \max_{1 \leq j \leq n} w_j\, (a_j + b_j), \tag{26}$$

each of the three holding for a reason of its own. We call them the M_1 threshold LB_1, the M_2 threshold LB_2 and the job threshold LB_3. The first two charge the workload of the prefix to the job of the prefix that leaves its machine last. That job carries a weight of at least w_j, and it completes its machine only after the whole workload of S has been processed there. Hence $WC^*_{\max} \geq w_j \sum_{h \in S} a_h$ and $WC^*_{\max} \geq w_j \sum_{h \in S} b_h$. The job threshold charges one job its own two operations, which run one after the other and neither of which can be left out, so $C^*_j \geq a_j + b_j$. As J_j carries the weight w_j itself, $w_j(a_j + b_j)$ is a lower bound on the optimum as well. Each bound is determined by the instance alone, and their largest is the reference value of the instance,

$$LB = \max\{LB_1, LB_2, LB_3\}, \tag{27}$$

so that $LB \le WC^*_{\max}$.

For the prefix S, the two bounds give $w_j \sum_{h\in S}(a_h+b_h) \le LB_1+LB_2 \le 2\,LB \le 2\,WC^*_{\max}$, and the claim follows from (23). □

4.4 FPTAS

4.4.1 Rounding the time coordinates

We now convert the exact dynamic program of Algorithm 2 into a fully polynomial-time approximation scheme by rounding the input data (Hall, 1994). One key point of this technique is the determination of valid lower and upper bounds for the coordinates to be rounded: each time coordinate ranges in a bounded interval, and rounding it to one of a fixed number of geometrically spaced values keeps the relative error within a single factor θ. The total processing times A_i, B_i of block $\mathcal{B}_i$ on M_1 and M_2, and its completion time C_i, are rounded. The weight L_i of the last job of block $\mathcal{B}_i$ is not rounded; following Algorithm 2, the positional constraint $L_i \in \{0, W_i\}$ is required for every block, and a state violating it is discarded.

Fix $0 < \epsilon < 1$ and let

$$\theta = 1 + \frac{\epsilon}{2n}, \qquad V_A = \sum_{j=1}^{n} a_j, \qquad V_B = \sum_{j=1}^{n} b_j, \qquad V = V_A + V_B.$$

For every block $\mathcal{B}_i$, the recurrence (21) preserves $A_i \le V_A$ and $B_i \le V_B$, since block $\mathcal{B}_i$ contains a subset of the jobs; moreover $C_i \le A_i + B_i \le V$. We therefore round the A dimension over $[0, V_A]$, the B dimension over $[0, V_B]$, and the C dimension over $[0, V]$. To this end, let

$$\Delta(x) = k = \lfloor \log_\theta x \rfloor \quad \text{for every } x \ge 1 \text{ with } \theta^k \le x < \theta^{k+1}, \qquad \text{and} \qquad \Delta(0) = -1.$$

All processing times are integral, so every value taken by a coordinate A_i, B_i, C_i is a non-negative integer. Hence any two values $y, y' \ge 1$ with $\Delta(y) = \Delta(y')$ satisfy $y \le \theta\, y'$; we call this the *interval-dominance* property, and it is the only fact about the rounding used in the analysis below.

The *label* of a state $\mathscr{H}^\epsilon = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K)$ is the tuple formed by applying Δ to its time coordinates, together with its block-end weights,

$$\ell(\mathscr{H}^\epsilon) = \big(\Delta(A_1), \dots, \Delta(A_K);\ \Delta(B_1), \dots, \Delta(B_K);\ \Delta(C_1), \dots, \Delta(C_K);\ L_1, \dots, L_K\big).$$

Thus two states share a label exactly when every coordinate A_i, B_i and C_i falls into the same box and their block-end weights coincide. Each dimension $x \in \{A, B, C\}$ contributes at most $v^x + 2$ distinct values of Δ, where $v^x = \lceil \log_\theta V_x \rceil$ and $V_C = V$, while the weight is not rounded and each L_i takes at most $n + 1$ values.

Algorithm 4: State-class FPTAS for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; accuracy ϵ.

Step 1. [Initialization] Set $\theta = 1 + \epsilon/(2n)$; set $Z^* \leftarrow +\infty$;
$\mathscr{H}_0^\epsilon \leftarrow \{(0,\dots,0;\ 0,\dots,0;\ 0,\dots,0;\ 0,\dots,0)\}$.

Step 2. [Generation] **for** $j = 1$ **to** n **do**

  for *each state* $(A_1,\dots,A_K; B_1,\dots,B_K; C_1,\dots,C_K; L_1,\dots,L_K) \in \mathscr{H}_{j-1}^\epsilon$ **do**

    for *each block* $i \in \mathcal{E}(J_j) = \{1,\dots,i^*\}$ **do**

      $\tilde{A}_i = A_i + a_j$, $\tilde{B}_i = B_i + b_j$, $\tilde{L}_i = w_j$

      $\tilde{C}_i = \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$

      [Label] Record the label $\ell(\tilde{\mathscr{H}}_j^\epsilon) =$
      $\big(\Delta(\tilde{A}_1),\dots,\Delta(\tilde{A}_K);\ \Delta(\tilde{B}_1),\dots,\Delta(\tilde{B}_K);\ \Delta(\tilde{C}_1),\dots,\Delta(\tilde{C}_K);\ \tilde{L}_1,\dots,\tilde{L}_K\big)$.

    end

    [Elimination] Among the candidate states sharing the same label, keep one of them and delete the others; set $\mathscr{H}_j^\epsilon$ to the kept states.

  end

end

Step 3. [Calculate] Same as Step 3 of Algorithm 2, applied to $\mathscr{H}_n^\epsilon$.

Step 4. [Result] Same as Step 4 of Algorithm 2.

Output: An approximate schedule σ.

The Elimination step keeps one representative per label. At iteration j, every state of $\mathscr{H}_{j-1}^\epsilon$ spawns one candidate for each eligible block $i \in \mathcal{E}(J_j) = \{1,\dots,i^*\}$, and each candidate is assigned its label. Two candidates with the same label are indistinguishable after rounding and carry the same block-end weights $L_1,\dots,L_K$, so the step keeps any one of them and discards the rest: the kept state satisfies $x \le \theta\,\tilde{x}$ in every time coordinate $x \in \{A, B, C\}$ of every block, relative to every candidate $\tilde{\mathscr{H}}^\epsilon$ it replaces, while its block-end weights are unchanged. Every label therefore carries exactly one state into $\mathscr{H}_j^\epsilon$, and the number of states never exceeds the number of labels.

Lemma 4.3. *Let* $\mathscr{H}_j = (A_1,\dots,A_K; B_1,\dots,B_K; C_1,\dots,C_K; L_1,\dots,L_K)$, $j = 1,\dots,n$, *be the states along an optimal trajectory of Algorithm 2, that is, the states recovered by backtracking from an optimal final state. For every* j, *Algorithm 4 keeps a state* $\hat{\mathscr{H}}_j^\epsilon = (\hat{A}_1,\dots,\hat{A}_K; \hat{B}_1,\dots,\hat{B}_K; \hat{C}_1,\dots,\hat{C}_K; L_1,\dots,L_K)$ *that carries the same block-end weights* $L_1,\dots,L_K$ *and satisfies*

$$\hat{A}_i \le A_i\,\theta^{j-1}, \quad \hat{B}_i \le B_i\,\theta^{j-1}, \quad \hat{C}_i \le C_i\,\theta^{j-1}, \quad (1 \le i \le K). \tag{28}$$

Proof. We argue by induction on j, from 1 to n.

Base case $j = 1$**.** Both algorithms generate their candidates for J_1 from the same zero state. Two candidates that place J_1 in different blocks differ in at least one time coordinate, so they carry different labels and no state is deleted. The sets $\mathscr{H}_1^\epsilon$ and $\mathscr{H}_1$ therefore coincide, and (28) holds with equality at $j = 1$.

Inductive step $j \geq 2$. Assume that (28) holds at $j-1$, and let i be the block to which $\mathscr{H}_{j-1}$ appends J_j in Algorithm 2. Algorithm 2 assigns a job only to a block i with $w \leq W_i$, so $w_j \leq W_i$. This constraint depends only on J_j and the block index, hence appending J_j to block i is available from $\hat{\mathscr{H}}^{\epsilon}_{j-1}$ as well. Write A'_i, B'_i and C'_i for the coordinates of $\mathscr{H}_{j-1}$. Applying the exact recurrences to $\hat{\mathscr{H}}^{\epsilon}_{j-1}$ and using $\theta^{j-2} \geq 1$ gives a candidate $\tilde{\mathscr{H}}^{\epsilon}_{j}$ with

$$\begin{aligned}
\tilde{A}_i &= \hat{A}_i + a_j \leq \big(A'_i + a_j\big)\theta^{j-2} = A_i\theta^{j-2},\\
\tilde{B}_i &= \hat{B}_i + b_j \leq \big(B'_i + b_j\big)\theta^{j-2} = B_i\theta^{j-2},\\
\tilde{C}_i &= \max\big\{\hat{C}_i + b_j,\ \hat{A}_i + a_j + b_j\big\}\\
&\leq \max\big\{C'_i + b_j,\ A'_i + a_j + b_j\big\}\theta^{j-2} = C_i\theta^{j-2},
\end{aligned}$$

where A_i, B_i and C_i are the coordinates of $\mathscr{H}_j$. For the remaining blocks $m \neq i$ the same transition leaves the coordinates unchanged, so that $\tilde{x}_m = \hat{x}_m \leq x'_m\theta^{j-2} = x_m\theta^{j-2}$ for $x \in \{A, B, C\}$. Hence $\tilde{x}_m \leq x_m\theta^{j-2}$ holds for every block $m = 1, \ldots, K$. The block-end weights $L_1, \ldots, L_K$ of the candidate are those of $\mathscr{H}_j$, because $L_i = w_j$ and the remaining block-end weights are unchanged.

The candidate $\tilde{\mathscr{H}}^{\epsilon}_{j}$ carries the label $\ell(\tilde{\mathscr{H}}^{\epsilon}_{j})$ assigned in Algorithm 4, and the algorithm keeps a state $\hat{\mathscr{H}}^{\epsilon}_{j} \in \mathscr{H}^{\epsilon}_{j}$ with that same label. Since the label records the block-end weights, $\hat{\mathscr{H}}^{\epsilon}_{j}$ carries $L_1, \ldots, L_K$ as well. Interval dominance states that two values with the same index Δ differ by at most a factor θ, so $y^x_m \leq \theta\,\tilde{x}_m$ for every $x \in \{A, B, C\}$ and every block $m = 1, \ldots, K$, where y^x_m denotes the coordinates of $\hat{\mathscr{H}}^{\epsilon}_{j}$. Together with $\tilde{x}_m \leq x_m\theta^{j-2}$, this yields (28) at j and completes the induction.

Every state of $\mathscr{H}^{\epsilon}_{n}$ stores all $3K$ time coordinates, so the induction covers the coordinates of the last block $\mathcal{B}_K$ as well. Storing these coordinates for every block, exactly as in Algorithm 2, rather than deriving them from the others, preserves the factor θ^{j-1} of (28): a derived value would accumulate the rounding errors of the remaining blocks. □

Theorem 4.3. *For every $0 < \epsilon < 1$, Algorithm 4 returns a fully polynomial-time approximation scheme with*

$$WC_{\max} \ \leq\ (1+\epsilon)\,WC^*_{\max}.$$

For fixed K, the algorithm runs in $O\Big(n^{4K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\epsilon^{3K}}\Big)$ time.

Proof. We first bound the value of the returned schedule, and then its running time.

Let $\mathscr{H}_n$ be the final state of the optimal trajectory of Algorithm 2, and let $\hat{\mathscr{H}}^{\epsilon}_{n} \in \mathscr{H}^{\epsilon}_{n}$ be the state guaranteed by Lemma 4.3 at $j = n$. By the lemma, the time coordinates of $\hat{\mathscr{H}}^{\epsilon}_{n}$ are within a factor θ^{n-1} of those of $\mathscr{H}_n$, and its block-end weights are those of $\mathscr{H}_n$, because the weights are not rounded. The final state $\mathscr{H}_n$ satisfies the positional constraint $L_i \in \{0, W_i\}$ of Algorithm 2, so every block of $\hat{\mathscr{H}}^{\epsilon}_{n}$ has block-end weight at most W_i.

Let $\hat{\sigma}$ and σ^* be the schedules obtained by concatenating the blocks of $\hat{\mathscr{H}}^{\epsilon}_{n}$ and $\mathscr{H}_n$, and let $C_i(\sigma)$ denote the completion time of the last job of block $\mathcal{B}_i$ in a schedule σ. By the

concatenation recurrence of Algorithm 1, $C_i(\sigma)$ is nondecreasing in every coordinate of the state and positively homogeneous. Hence $C_i(\hat{\sigma}) \le \theta^{n-1} C_i(\sigma^*)$ for every i. Since every block of $\hat{\mathscr{H}}_n^{\epsilon}$ has block-end weight at most W_i, Lemma 4.1 yields

$$WC_{\max}(\hat{\sigma}) = \max_{1\le i\le K} W_i\, C_i(\hat{\sigma}) \le \theta^{n-1} \max_{1\le i\le K} W_i\, C_i(\sigma^*) = \theta^{n-1}\, WC^*_{\max}.$$

The factor θ^{n-1} is at most $1+\epsilon$ for $0<\epsilon<1$. Because $1+x\le e^x$ for every real x,

$$\theta^{n-1} = \left(1+\frac{\epsilon}{2n}\right)^{n-1} \le e^{\frac{(n-1)\epsilon}{2n}} < e^{\epsilon/2},$$

where the strict inequality uses $(n-1)/(2n) < 1/2$. It is therefore enough to check that $e^{\epsilon/2} \le 1+\epsilon$ on $[0,1]$. The function $g(\epsilon) = 1+\epsilon-e^{\epsilon/2}$ satisfies $g(0)=0$ and

$$g'(\epsilon) = 1 - \tfrac{1}{2}e^{\epsilon/2} \ge 1 - \tfrac{1}{2}e^{1/2} > 0 \qquad (0\le\epsilon\le 1),$$

because $e^{\epsilon/2} \le e^{1/2} < 2$ on that interval. Hence g is increasing, so $g(\epsilon)\ge 0$ and $e^{\epsilon/2}\le 1+\epsilon$. Consequently $\theta^{n-1}\le 1+\epsilon$, and the schedule returned by Algorithm 4, being no worse than $\hat{\sigma}$, has a value of at most $(1+\epsilon)\,WC^*_{\max}$.

We now bound the running time. At each level j, Algorithm 4 keeps at most one state per label, and the number of labels is $(v^A+2)^K(v^B+2)^K(v^C+2)^K(n+1)^K$. The rounded time coordinates contribute $(v^A{+}2)^K(v^B{+}2)^K(v^C{+}2)^K$, which is at most $(v^C{+}2)^{3K}$ since $v^A, v^B \le v^C$, and each of the K block-end weights takes at most $n+1$ values. Each state generates at most K candidates, one per eligible block, in $O(1)$ time each. The final evaluation of the states of $\mathscr{H}^{\epsilon,n}$ costs $O(K)$ per state. The running time is therefore $O\big(nK\,(v^C{+}2)^{3K}(n{+}1)^K\big)$. Since $\ln(1+\epsilon/(2n)) \ge \epsilon/(4n)$ for $0<\epsilon<1$, we have

$$v^C = \left\lceil \frac{\ln V}{\ln\theta} \right\rceil = O\left(\frac{n\log V}{\epsilon}\right).$$

For constant K, the bound reduces to $O(n^{4K+1}K(\log V)^{3K}/\epsilon^{3K})$, which is polynomial in n, $\log V$ and $1/\epsilon$. □

4.4.2 Weight rounding

The FPTAS above rounds the time coordinates A_i, B_i, C_i and assumes that the number K of distinct weights is fixed. An analogous geometric rounding applies to the weights themselves when their ratio is bounded. Let $W_{\min} = \min_j w_j$, $W_{\max} = \max_j w_j$, and suppose $W_{\max}/W_{\min} \le c$ for a constant c. Partition the interval $[W_{\min}, W_{\max}]$ into

$$J_0 = [W_{\min}, W_{\min}\theta), \qquad J_r = [W_{\min}\theta^r, W_{\min}\theta^{r+1}) \ \ (1\le r\le q-1), \qquad J_q = [W_{\min}\theta^q, W_{\max}],$$

with $q = \big\lceil \log_\theta(W_{\max}/W_{\min}) \big\rceil$, and round each weight w_j upward to the right endpoint of its interval,

$$\hat{w}_j = W_{\min}\,\theta^{\,r_j+1}, \qquad r_j = \left\lfloor \log_\theta \frac{w_j}{W_{\min}} \right\rfloor,$$

so that $w_j \le \hat{w}_j \le \theta\, w_j$. The rounded weights take only $q+1 = O\big(\log_\theta(W_{\max}/W_{\min})\big)$ distinct values, so jobs whose weights differ by a factor at most θ merge into a single class.

Lemma 4.4. *Let $\hat{w}_j$ be the rounded weights defined above, and let $\hat{\sigma}$ be an optimal schedule of the instance with weights $\hat{w}_j$, whose optimal value is $\hat{W}C^*_{\max}$. Then*

$$WC_{\max}(\hat{\sigma}) \le \theta\, WC^*_{\max}. \tag{29}$$

Proof. By construction $w_j \le \hat{w}_j \le \theta w_j$ for every job J_j. Since the maximum is monotone, every schedule σ satisfies

$$WC_{\max}(\sigma) = \max_{1\le j\le n} w_j C_j(\sigma) \le \max_{1\le j\le n} \hat{w}_j C_j(\sigma) = \hat{W}C_{\max}(\sigma) \le \theta\, WC_{\max}(\sigma).$$

Let π^* be an optimal schedule of the original instance. The rounded problem is solved optimally by $\hat{\sigma}$, and π^* is feasible for it, so

$$WC_{\max}(\hat{\sigma}) \le \hat{W}C_{\max}(\hat{\sigma}) = \hat{W}C^*_{\max} \le \hat{W}C_{\max}(\pi^*) \le \theta\, WC_{\max}(\pi^*) = \theta\, WC^*_{\max},$$

which is (29). □

Hence the weight rounding incurs a relative error of at most $\theta = 1 + \epsilon/(2n) \le 1 + \epsilon$, exactly as in the completion-time rounding.

5 Heuristic Algorithms

The exact dynamic program of Section 4 is pseudo-polynomial, and the running time of the FPTAS of Section 4.4 is exponential in the number K of distinct weights. For large instances, fast heuristics are therefore of practical interest. In this section we present three heuristics for $F2 \parallel WC_{\max}$: an adaptation of the NEH algorithm (Puka et al., 2024), an ant colony optimization (ACO) algorithm (Zhang et al., 2023), and a genetic algorithm (Reeves, 1995).

Throughout this section, the objective of a permutation $\sigma = (\sigma(1), \ldots, \sigma(k))$ of k jobs is evaluated by (1), so that

$$WC_{\max}(\sigma) = \max_{1\le t\le k} w_{\sigma(t)}\, C_{\sigma(t)},$$

which is computed in $O(k)$ time.

5.1 NEH

Constructive heuristics are attractive for $F2 \parallel WC_{\max}$ because they produce a complete schedule in a single pass, without the tuning effort of population-based metaheuristics. The NEH heuristic (Puka et al., 2024) is among the most effective constructive rules for permutation flow shops, and its core idea is to build a schedule by incrementally inserting jobs into a growing partial sequence, each time choosing the position that best serves the objective.

Algorithm 5: NEH heuristic for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

Step 1. [Sequence] Sort the jobs by non-increasing $a_j + b_j$, breaking ties by w_j; let $J_1, \ldots, J_n$ be the resulting sequence.

Step 2. [Initialization] $\sigma \leftarrow (J_1)$.

Step 3. [Insertion] **for** $k = 2$ **to** n **do**

  $\sigma \leftarrow$ the sequence obtained by inserting J_k into σ at a position minimizing $WC_{\max}$.

end

Output: Schedule σ.

In the maximum weighted completion-time objective, two properties are particularly important. First, jobs with large total processing time $a_j + b_j$ tend to push the whole makespan forward and therefore should appear early so as not to delay other jobs. Second, given a fixed makespan, jobs with larger weight w_j benefit more from occupying earlier positions, because their weighted completion time decreases. These two properties suggest a greedy priority in which large-$a_j + b_j$ jobs are placed first and large-w_j jobs are used as a tie-breaker.

We adapt the NEH framework as follows. The jobs are first sorted by non-increasing $a_j + b_j$, breaking ties by non-increasing w_j, to obtain an initial sequence $J_1, \ldots, J_n$. The schedule starts with the single job J_1. For each $k = 2, \ldots, n$, the next job J_k is inserted into every one of the k possible positions of the current partial schedule. Each candidate sequence is evaluated by (1), and the position that yields the smallest $WC_{\max}$ is retained. The pseudo-code is given in Algorithm 5.

Because the partial schedule already contains $k - 1$ jobs, the cost of evaluating all k insertions at step k is $O(k^2)$ using the recurrence (1); the total running time is therefore $O(n^3)$ (Theorem 5.1). The heuristic also produces a feasible permutation schedule by construction, requires no hyperparameter selection, and can serve either as a fast standalone solver for large instances or as a warm-start seed for the population-based heuristics described below.

Theorem 5.1. *Algorithm 5 runs in $O(n^3)$ time.*

Proof. At step k, the job J_k is inserted into each of the k positions of a partial schedule of $k - 1$ jobs, and each candidate sequence is evaluated in $O(k)$ time by the recurrence above. Hence step k costs $O(k^2)$, and the total running time is $O\big(\sum_{k=1}^{n} k^2\big) = O(n^3)$. □

5.2 Ant Colony Optimization

Population-based metaheuristics can explore a much larger set of schedules than constructive heuristics, but their effectiveness depends on how well the search bias matches the problem structure. Ant colony optimization (Zhang et al., 2023) is a constructive popula-

tion metaheuristic in which a colony of artificial ants incrementally builds schedules, and the collective experience is stored in pheromone trails that guide subsequent ants toward promising job orderings.

For $F2 \parallel WC_{\max}$, the relative order of jobs is more important than an absolute assignment: placing a heavy job too late inflates the objective dramatically, while keeping it early usually helps regardless of the exact surrounding permutation. This makes pairwise preference information a natural search bias. ACO captures such preferences through a pheromone matrix τ_{ij}, where a larger value encourages job J_j to be scheduled after job J_i, thereby biasing the colony toward front-heavy schedules.

We adapt the standard ACO framework as follows. Initially all pheromone values are set to $\tau_{ij} = 1/n$. In every iteration, each of the m ants constructs a permutation σ_k by repeatedly selecting, from the set U of unscheduled jobs, the job J_j to be placed at the next position with probability

$$p_{jt} = \frac{\tau_j^{\alpha} \cdot \eta_{jt}^{\beta}}{\sum_{l \in U} \tau_l^{\alpha} \cdot \eta_{lt}^{\beta}}.$$

Here α and β control the relative influence of pheromone and heuristic information η_{jt}, which we set to $\eta_{jt} = w_j/(a_j + b_j + 1)$ so that jobs with large weight and small total processing time are preferred. After all ants have built their schedules, the best schedule σ^* found so far is identified, and the pheromone trails are updated according to

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \rho\, \Delta\tau_{ij},$$

where $\rho \in (0, 1)$ is the evaporation rate and $\Delta\tau_{ij} = Q/Z^*$ if the edge (i, j) appears in σ^*, and 0 otherwise. The process is repeated until a stopping criterion is met. The pseudo-code is given in Algorithm 6.

Because ants construct complete schedules probabilistically, ACO can escape the restricted search region of deterministic heuristics. The pheromone mechanism also provides a compact, increasingly focused memory of good pairwise orders, so the algorithm naturally intensifies around high-quality regions while the stochastic construction preserves diversification. Moreover, each schedule is evaluated in $O(n)$ time by (1), making each iteration computationally lightweight and well suited to large instances.

Algorithm 6: Ant Colony Optimization for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); parameters α, β, ρ, Q; number of ants m.

Step 1. [Initialization] Initialize pheromone matrix $\tau_{ij} \leftarrow 1/n$ for all i, j; set best solution σ^*, $Z^* \leftarrow +\infty$.
Step 2. [Construction] **for** *ant* $k = 1$ **to** m **do**
    Build a permutation σ_k by probabilistically selecting jobs:
    for *position* $t = 1$ **to** n **do**
        Calculate probabilities $p_{jt} = \frac{\tau_j^{\alpha} \cdot \eta_{jt}^{\beta}}{\sum_{l \in U} \tau_l^{\alpha} \cdot \eta_{lt}^{\beta}}$ where U is unselected jobs;
        Select job j with probability p_{jt} and set $\sigma_k(t) = j$;
    end
    Evaluate $Z_k = WC_{\max}(\sigma_k)$;
    if $Z_k < Z^*$ **then**
        $\sigma^* \leftarrow \sigma_k$, $Z^* \leftarrow Z_k$
    end
end
Step 3. [Update Pheromone] **for** *each edge* (i, j) **do**
    $\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \rho \cdot \Delta\tau_{ij}$ where $\Delta\tau_{ij} = Q/Z^*$ if (i, j) in σ^*, else 0;
end
Step 4. [Termination] **if** *stopping criterion not met* **then**
    go to Step 2.
end

Output: Best schedule σ^*.

5.3 Genetic Algorithm

The objective $\max_j w_j C_j$ is a bottleneck criterion: the value of a schedule is determined by the single *critical* job that attains the maximum, and an ordering can be improved only by lowering that job's completion time, or by shortening completion times across the board. A genetic algorithm for $F2 \parallel WC_{\max}$ should therefore not treat all jobs symmetrically. Following the genetic algorithm (Reeves, 1995) for flow-shop sequencing, we design its operators around the weight classes and the block structure of Section 4: high-weight jobs should occupy early positions (their weighted completion must be kept small), while the makespan should stay low so that no job is delayed unnecessarily.

A chromosome is a permutation $\sigma = (\sigma(1), \ldots, \sigma(n))$, where the gene at position t is the job scheduled t-th. Permutation encoding is adopted because it always represents a feasible permutation schedule: every job occurs exactly once, so no repair operator is needed to restore feasibility after crossover or mutation. To decode a chromosome, both machines process the jobs in the order $\sigma(1), \ldots, \sigma(n)$; the completion time $C_{\sigma(t)}$ of the t-th job is obtained from the closed form (1), which is evaluated in $O(n)$ time using the prefix sums

$A_t = \sum_{h=1}^{t} a_{\sigma(h)}$ and $B_t = \sum_{h=1}^{t} b_{\sigma(h)}$, and the objective value is

$$WC_{\max}(\sigma) = \max_{1 \le t \le n} w_{\sigma(t)} C_{\sigma(t)}.$$

The fitness $\text{fitness}(\sigma) = 1/WC_{\max}(\sigma)$ is used only to compare individuals, so its absolute scale is irrelevant.

Random starting populations waste most of the search effort. The initial population P_0 of size N is therefore seeded with the three orderings that already realize the two structural ingredients of a good schedule—small makespan and early heavy jobs:

- the Johnson schedule σ^{J} produced by Johnson's rule (Johnson, 1954), which minimizes the makespan and thereby tends to shorten every completion time;
- the weight-descending schedule σ^{W}, obtained by ordering jobs by non-increasing w_j with ties broken by Johnson's rule, which directly attacks the bottleneck by giving heavy jobs the smallest possible completion times;
- the schedule σ^{NEH} returned by the NEH heuristic (Algorithm 5).

The remaining $N-3$ individuals are uniform random permutations, which supply the diversity needed to escape these deterministic starting points and to explore regions of the search space that no constructive rule reaches. The population size is $N = \Theta(n)$: large enough to retain the block structure present in the seeds, yet small enough that a generation, whose cost is dominated by evaluating N individuals in $O(n)$ time each, remains $O(Nn)$.

Binary tournament selection is used to form the mating pool. In each round, two individuals are drawn uniformly at random from the current population P_{g-1}, the fitter of the two wins and is copied into the mating pool M_g, and both are returned to the population (ties are broken arbitrarily); this is repeated until M_g contains N individuals. Tournament selection is preferred over roulette-wheel selection because it depends only on the *ranking* of fitness values rather than on their absolute magnitudes, so it remains selective even as the population converges and the fitness values of near-optimal schedules become very close.

Crossover recombines two parent schedules into offspring that inherit both the relative order and the absolute positions of jobs—the two ingredients a permutation schedule is built from. We use the partially mapped crossover (PMX), the standard operator for permutation-encoded genetic algorithms. Two cut points $p_1 < p_2$ are drawn uniformly at random, and the middle segment $\pi_1[p_1..p_2]$ is copied verbatim into the child. The remaining positions are filled from the second parent π_2; whenever a gene of π_2 already occurs in the child, it is replaced through the mapping induced by the middle segment, so the child is always a valid permutation. PMX transfers contiguous blocks of jobs, which realize the front-heavy structure identified in Section 4, while still recombining the two parents' orders on the outer positions. Swapping the roles of π_1 and π_2 yields the second child.

Algorithm 7: Partially Mapped Crossover (PMX) for $F2 \parallel WC_{\max}$

Input: Parents π_1, π_2; cut points $p_1 < p_2$.

Step 1. [Copy segment] Set $\pi'[t] \leftarrow \pi_1[t]$ for $t = p_1, \dots, p_2$.
Step 2. [Fill outer positions] **for** $t = 1$ **to** n, $t \notin [p_1, p_2]$ **do**
  $j \leftarrow \pi_2[t]$
  while j *occurs in* π' **do**
    $j \leftarrow \pi_2[\pi_1^{-1}(j)]$ `// follow the PMX mapping`
  end
  $\pi'[t] \leftarrow j$
end

Output: Child π'.

Algorithm 8: Critical-Job Mutation for $F2 \parallel WC_{\max}$

Input: Schedule σ; mutation probability p_m.

Step 1. [Identify] Evaluate $WC_{\max}(\sigma)$ by (1); let j^* be a job attaining the maximum and let t^* be its position in σ.
Step 2. [Insertion search] **for** $r = 1$ **to** $t^* - 1$ **do**
  Form $\sigma^{(r)}$ by moving j^* to position r; evaluate $WC_{\max}(\sigma^{(r)})$.
end
Step 3. **if** $\min_r WC_{\max}(\sigma^{(r)}) < WC_{\max}(\sigma)$ **then**
  $\sigma \leftarrow \sigma^{(r^*)}$, where $r^* = \arg\min_r WC_{\max}(\sigma^{(r)})$
else
  if *rand*() $< p_m$ **then**
    Swap two jobs of σ lying in different weight classes.
  end
end

Output: Improved schedule σ.

Mutation aims at the job that currently decides the objective. Let j^* be the critical job attaining $WC_{\max}(\sigma)$ at position t^*. Moving j^* to an earlier position $r < t^*$ can only decrease its own completion time (its machine-2 work is unchanged while it may start earlier), so the weighted completion $w_{j^*}C_{j^*}$—currently the largest—tends to decrease. The mutation therefore tries every insertion of j^* into a position $r < t^*$, evaluates the resulting permutation by (1), and keeps the best; this is a cheap, targeted hill-climbing step. If no such insertion improves the value, a random swap of two jobs from different weight classes is performed with probability p_m to maintain diversity. Applied to every offspring, and combined with the elitist replacement, this operator acts as a local search that monotonically improves the incumbent without being trapped by the deterministic seeds.

Algorithm 9: Genetic Algorithm for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$; population size N; generations G; probabilities p_c, p_m; elitism size λ.

Step 1. [Initialization] Build P_0 from the structured schedules $\sigma^{\mathrm{J}}, \sigma^{\mathrm{W}}, \sigma^{\mathrm{NEH}}$ and $N-3$ random permutations; evaluate all individuals; set σ^* to the best.

Step 2. [Evolution] **for** $g = 1$ **to** G **do**

  Selection Form the mating pool M_g from P_{g-1} by binary tournament selection.

  Crossover while $|O| < N$ **do**

    Draw two parents from M_g; with probability p_c apply Algorithm 7 to obtain two children, otherwise copy the parents into the offspring set O.

  end

  Mutation Apply Algorithm 8 to each offspring with probability p_m; evaluate every child by (1).

  Replacement $P_g \leftarrow$ the best λ individuals of P_{g-1} plus the $N-\lambda$ best offspring.

  Update If the best individual of P_g improves on σ^*, replace σ^* by it.

end

Step 3. [Local search] Apply Algorithm 8 to σ^* until it yields no improvement.

Output: Best schedule σ^*.

Genetic Algorithm Combining the ingredients above gives Algorithm 9. Elitism preserves the best λ solutions of each generation, and the final critical-job mutation plays the role of a local search.

6 Experiments

All methods were coded in Python 3.13 and run on a machine with an Intel Core Ultra 5 230F processor and 16 GB of memory, under Windows 11. The MILP model was solved with Gurobi 13.0 on a single thread, so that the reported times are comparable across configurations.

6.1 Experimental setup

Every algorithm was run once on every instance, and no method was handicapped by a smaller budget than another, since all of them evaluate the objective by the same recurrence (1). The exact methods and the heuristics are read on different quantities. An exact method is read on the number of instances it solves to optimality, a heuristic on its relative percentage gap against a theoretical lower bound, and both on the CPU time they require. The gap of method A is

$$\mathrm{gap}_A = \frac{z_A - z_{\mathrm{base}}}{z_{\mathrm{base}}}, \tag{30}$$

where z_A is the objective value of the schedule it returned and z_{base} is that lower bound, the reference value LB of (27): the largest of the three thresholds LB_1, LB_2 and LB_3 of Section 4.3, each of which the instance determines on its own, so that no schedule of a method enters the denominator of its gap.

The parameters of both metaheuristics were fixed by a calibration run before the study. The ant colony optimization uses 10 ants, $\alpha = 1$, $\beta = 30$, $\rho = 0.1$ and 100 iterations. The genetic algorithm uses a population of $N = n$ individuals, $G = 10$ generations, $p_c = 0.9$, $p_m = 0.1$ and an elitism size of $\lambda = N/10$.

The instances were randomly generated at two scales. The processing times a_j and b_j were drawn independently for every job and every instance from a discrete uniform range, whose upper end the configuration fixes. The small scale takes the three narrow ranges $[1, 20]$, $[1, 40]$ and $[1, 60]$, the large one the two wide ranges $[1, 100]$ and $[1, 200]$, so that the width of the range is itself read as a factor at both scales. The weights of the small scale are the geometric ladder $1, 10, \ldots, 10^{K-1}$, whose length K the configuration fixes at 2 and 3: the K classes are then decades apart, so that the gap between two of them stays wide whatever their number, while the range of the processing times is what varies underneath. On the large scale the weights are unrestricted and each of them is drawn uniformly from $\{1, \ldots, n\}$, so that their number is bounded by the number of jobs rather than by a fixed K. The two scales of Table 1 organise these factors. The small scale contains 360 instances, three sizes crossed with the two weight counts and the three ranges. The large scale contains 120 instances, three sizes crossed with the two ranges, at the single unrestricted weight regime. Each configuration was replicated over 20 independent instances, and every figure below is the average of those replications. Every method was given the same time limit of 600 s. A method that reaches the limit is recorded with the value it has attained rather than omitted. For every configuration the tables report the number of optimal solutions of each exact method and, for each heuristic, the minimum, the mean and the maximum of its gap, together with the mean objective value and the mean CPU time of both.

The algorithms of the previous sections were compared on these instances. The MILP model of Section 3 and the dynamic program of Algorithm 2 were applied to every small instance, and the three heuristics of Section 5 to every large one. The two exact methods were compared on the small scale, and the tables that report them are read for how sensitive they are to the number of jobs and to the number of distinct weights. The heuristics were compared on the large scale, where the factor read is the instance size.

Table 1: Each configuration is replicated over 20 independent instances

Scale	n	Weights	Methods compared
Small	20, 40, 60	$K = 2$, $K = 3$	MILP, DP
Large	600, 800, 1000	All weights	NEH, ACO, GA

The tables and the figure below carry the results, and each is read for one question. Tables 2 to 5 report the small scale, one table per exact method and weight setting, and are read for the number of instances the method settles to optimality and its running time as the processing-time range widens. Tables 6 to 8 report the large scale, one table per heuristic, and are read for the gap and the running time of each heuristic under the instance size. Figure 4 collects the mean gaps of every heuristic against the instance size. Every entry of these tables is a placeholder until the corresponding run has been made, and the figure is left empty for the same reason.

6.2 Results

Table 2 shows [TBD].

Table 2: MILP on the small scale, $K = 2$. "opt." count optimality.

n	a_j, b_j range	$WC_{\max}$	opt.	time (s)
20	$[1, 20]$	1243.0	20/20	4.63
		(640.0, 1810.0)		(0.68, 16.91)
	$[1, 40]$	2217.5	20/20	9.98
		(1680.0, 3240.0)		(1.24, 70.69)
	$[1, 60]$	3213.0	20/20	8.29
		(2400.0, 4760.0)		(0.53, 35.87)
40	$[1, 20]$	2225.0	20/20	116.14
		(1530.0, 2720.0)		(8.66, 432.18)
	$[1, 40]$	4287.0	18/20	285.83
		(2780.0, 6860.0)		(20.56, 600.04)
	$[1, 60]$	6929.0	16/20	296.91
		(4690.0, 9190.0)		(12.76, 600.07)
60	$[1, 20]$	3155.5	4/20	546.16
		(2260.0, 4430.0)		(181.46, 600.12)
	$[1, 40]$	6480.0	2/20	579.40
		(4650.0, 8480.0)		(277.87, 600.12)
	$[1, 60]$	9403.0	2/20	593.78
		(6880.0, 12230.0)		(536.91, 600.10)

Table 3 shows [TBD].

Table 3: MILP on the small scale, $K = 3$. "opt." count optimality.

n	a_j, b_j range	$WC_{\max}$	opt.	time (s)
20	$[1, 20]$	8715.0	20/20	3.90
		(2700.0, 15800.0)		(0.09, 18.06)
	$[1, 40]$	17245.0	20/20	4.51
		(7300.0, 34900.0)		(0.18, 12.14)
	$[1, 60]$	24925.0	20/20	8.84
		(10500.0, 38700.0)		(0.10, 29.39)
40	$[1, 20]$	16040.0	17/20	237.02
		(10500.0, 22900.0)		(26.44, 600.05)
	$[1, 40]$	30735.0	16/20	308.71
		(16800.0, 47900.0)		(40.90, 600.05)
	$[1, 60]$	45295.0	12/20	416.23
		(28000.0, 60000.0)		(66.45, 600.06)
60	$[1, 20]$	22570.0	1/20	570.74
		(14600.0, 30900.0)		(15.82, 600.11)
	$[1, 40]$	45080.0	1/20	595.21
		(31400.0, 57700.0)		(502.64, 600.14)
	$[1, 60]$	66445.0	1/20	583.66
		(48100.0, 88600.0)		(271.61, 600.11)

Table 4 shows **[TBD]**.

Table 4: DP on the small scale, $K = 2$. "opt." count optimality.

n	a_j, b_j range	$WC_{\max}$	opt.	time (s)
20	$[1, 20]$	1243.0	20/20	0.01
		(640.0, 1810.0)		(0.00, 0.07)
	$[1, 40]$	2217.5	20/20	0.02
		(1680.0, 3240.0)		(0.00, 0.08)
	$[1, 60]$	3213.0	20/20	0.02
		(2400.0, 4760.0)		(0.00, 0.11)
40	$[1, 20]$	2225.0	20/20	0.84
		(1530.0, 2720.0)		(0.11, 3.26)
	$[1, 40]$	4287.0	20/20	4.73
		(2780.0, 6860.0)		(0.03, 20.43)
	$[1, 60]$	6929.0	20/20	4.81
		(4690.0, 9190.0)		(0.29, 19.60)
60	$[1, 20]$	3155.5	20/20	7.93
		(2260.0, 4430.0)		(1.94, 20.45)
	$[1, 40]$	6480.0	20/20	29.67
		(4650.0, 8480.0)		(1.38, 81.64)
	$[1, 60]$	9402.5	20/20	61.25
		(6880.0, 12230.0)		(11.67, 215.43)

Table 5 shows **[TBD]**.

Table 5: DP on the small scale, $K = 3$. "opt." count optimality.

n	a_j, b_j range	$WC_{\max}$	opt.	time (s)
20	$[1, 20]$	8715.0	20/20	2.81
		(2700.0, 15800.0)		(0.01, 19.51)
	$[1, 40]$	17245.0	20/20	4.12
		(7300.0, 34900.0)		(0.00, 33.16)
	$[1, 60]$	24925.0	20/20	3.68
		(10500.0, 38700.0)		(0.04, 23.42)
40	$[1, 20]$	**[TBD]**	**[TBD]**	**[TBD]**
		(**[TBD]**, **[TBD]**)		(**[TBD]**, **[TBD]**)
	$[1, 40]$	**[TBD]**	**[TBD]**	**[TBD]**
		(**[TBD]**, **[TBD]**)		(**[TBD]**, **[TBD]**)
	$[1, 60]$	**[TBD]**	**[TBD]**	**[TBD]**
		(**[TBD]**, **[TBD]**)		(**[TBD]**, **[TBD]**)
60	$[1, 20]$	**[TBD]**	**[TBD]**	**[TBD]**
		(**[TBD]**, **[TBD]**)		(**[TBD]**, **[TBD]**)
	$[1, 40]$	**[TBD]**	**[TBD]**	**[TBD]**
		(**[TBD]**, **[TBD]**)		(**[TBD]**, **[TBD]**)
	$[1, 60]$	**[TBD]**	**[TBD]**	**[TBD]**
		(**[TBD]**, **[TBD]**)		(**[TBD]**, **[TBD]**)

Table 6 shows **[TBD]**.

Table 6: NEH on the large scale.

n	a_j, b_j range	$WC_{\max}$	gap (%)[1]	time (s)
600	$[1, 100]$	5299345.8	13.70	7.75
		(4915356.0, 5763520.0)	(9.77, 18.41)	(7.66, 7.91)
	$[1, 200]$	10773490.0	15.40	7.57
		(10041360.0, 11733118.0)	(11.12, 19.38)	(7.51, 7.67)
800	$[1, 100]$	9340046.4	13.07	17.95
		(8507700.0, 9939672.0)	(9.26, 16.22)	(17.74, 18.18)
	$[1, 200]$	19285449.1	15.25	18.08
		(17865632.0, 20488644.0)	(10.16, 19.98)	(17.90, 18.49)
1000	$[1, 100]$	14499435.8	12.84	33.25
		(13467423.0, 15722994.0)	(10.08, 15.70)	(32.95, 33.98)
	$[1, 200]$	30026955.6	14.24	33.24
		(28436556.0, 31595004.0)	(9.62, 16.84)	(33.01, 33.62)

Table 7 shows [TBD].

Table 7: ACO on the large scale.

n	a_j, b_j range	$WC_{\max}$	gap (%)[1]	time (s)
600	$[1, 100]$	7263859.7	55.81	42.89
		(6296500.0, 8083060.0)	(46.21, 65.80)	(42.28, 43.70)
	$[1, 200]$	14511034.8	55.49	45.67
		(13726396.0, 15800155.0)	(44.56, 67.14)	(43.27, 47.35)
800	$[1, 100]$	12950359.2	56.80	81.47
		(11901736.0, 13693286.0)	(50.03, 68.35)	(79.33, 83.14)
	$[1, 200]$	26079832.5	55.85	82.46
		(23635392.0, 28648320.0)	(47.26, 64.07)	(80.04, 83.79)
1000	$[1, 100]$	20232817.4	57.43	117.63
		(18225336.0, 21700875.0)	(49.36, 64.60)	(117.15, 118.17)
	$[1, 200]$	41142901.0	56.54	119.09
		(38543352.0, 44123938.0)	(48.20, 64.97)	(118.76, 119.68)

Table 8 shows [TBD].

Table 8: GA on the large scale.

n	a_j, b_j range	$WC_{\max}$	gap (%)[1]	time (s)
600	$[1, 100]$	4734261.2	1.50	33.15
		(4323798.0, 5229332.0)	(0.20, 3.80)	(31.07, 34.90)
	$[1, 200]$	9558743.8	2.32	34.30
		(8767084.0, 10833685.0)	(0.55, 6.16)	(32.83, 35.26)
800	$[1, 100]$	8407268.4	1.77	78.75
		(7815087.0, 9098128.0)	(0.01, 4.15)	(76.08, 83.06)
	$[1, 200]$	17058653.9	1.90	75.93
		(15595944.0, 18219495.0)	(0.08, 5.34)	(72.64, 77.76)
1000	$[1, 100]$	13021374.1	1.31	140.01
		(11681496.0, 14290368.0)	(0.05, 3.34)	(134.22, 147.10)
	$[1, 200]$	26618259.1	1.23	141.60
		(24864444.0, 28932453.0)	(0.12, 3.42)	(134.46, 145.03)

6.3 Discussion

The dynamic program dominated the MILP model at the small scale. The model proved optimality on [TBD]of the 360 small instances, [TBD]at $n = 20$, [TBD]at $n = 40$ and

[TBD]at $n = 60$, which validates both implementations. The two methods separated on time long before they separated on value: the dynamic program was faster by a factor of [TBD]already at $n = 20$ and by a factor of [TBD]at $n = 60$, where the model needed [TBD] min on average and the dynamic program [TBD] min. The [TBD]instances the model did not close within its time limit are the only ones whose optimality it does not establish. There it reports the incumbent it had reached, and the dynamic program returned the same value on each of them, so those incumbents were optimal as well.

At the large scale every gap is non-negative by construction, and it measures the distance to the reference as much as the quality of the method: the reference LB of (27) caps the optimum by three prefix bounds, and it is tight only where the heaviest jobs are also the earliest ones, which no schedule of a heuristic is bound to achieve. What the gaps do order is the methods themselves, because the three of them are measured against the same z_{base} on every instance. The genetic algorithm returned the smallest gap at each setting, [TBD]points below NEH and [TBD]points below the ant colony optimization on average, and it did so on all 60 instances of the scale, where NEH attained the best value on [TBD]of them and the ant colony optimization on [TBD]. That advantage is what a population-based search with crossover and mutation built around the weight classes and the critical job would be expected to produce, and it is small against the running time it costs, [TBD]times that of the NEH-type rule.

The ant colony optimization returned the largest gap at every setting, [TBD]points above the genetic algorithm. The reason is structural rather than incidental. Its heuristic information is $\eta_j = w_j/(a_j + b_j + 1)$, so as the exponent β grows the construction approaches the weight-descending order and ignores the processing times, and the measured gap is smallest at an intermediate β and grows on both sides of it. We report this negative result rather than tuning the colony until it disappears, because it delimits the criterion for which the metaheuristic is worth its cost.

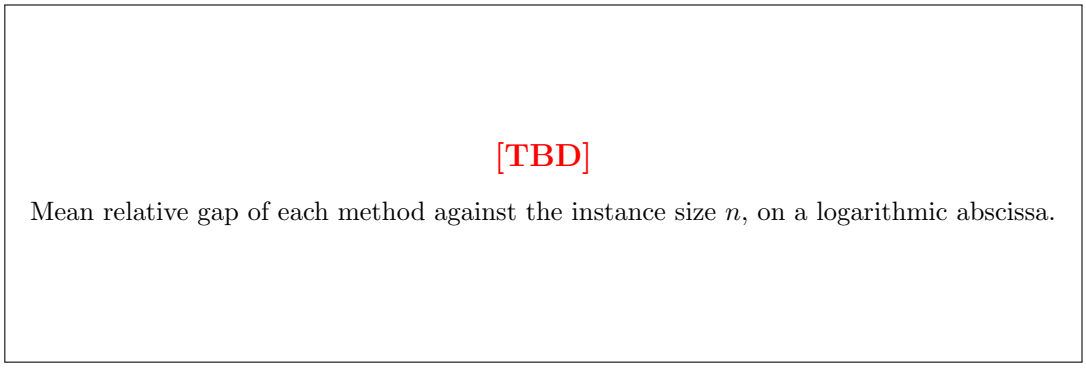


Figure 4: Mean relative gap as a function of the instance size.

7 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling*. Wiley, New York.

Guo, L., Xia, F., and Zhang, X. (2026). Single machine scheduling to minimize maximum earliness/tardiness cost with job rejection. *CCF Transactions on High Performance Computing*, 8(3):314–330.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Li, W. (2015). Best-possible online algorithms for single machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 32(4):1550030.

Lu, L., Zuo, L., Zhang, L., and Ou, J. (2025). Order acceptance and scheduling with weighted makespan. *4OR*, 23(3):247–267.

Panwalkar, S. S. and Koulamas, C. (2015). On equivalence between the proportionate flow shop and single-machine scheduling problems. *Naval Research Logistics*, 62(7):595–603.

Panwalkar, S. S., Smith, M. L., and Koulamas, C. (2013). Review of the ordered and proportionate flow shop scheduling research. *Naval Research Logistics*, 60(1):46–55.

Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, 6 edition.

Puka, R., Skalna, I., Duda, J., and Stawowy, A. (2024). Deterministic constructive vn-NEH+ algorithm to solve permutation flow shop scheduling problem with makespan criterion. *Computers & Operations Research*, 162:106473.

Reeves, C. R. (1995). A genetic algorithm for flowshop sequencing. *Computers & Operations Research*, 22(1):5–13.

Shabtay, D. and Gerstl, E. (2024). Coordinating scheduling and rejection decisions in a two-machine flow shop scheduling problem. *European Journal of Operational Research*, 316(3):887–898.

Shakhlevich, N., Hoogeveen, H., and Pinedo, M. (1998). Minimizing total weighted completion time in a proportionate flow shop. *Journal of Scheduling*, 1(3):157–168.

Sun, R. (2024). Randomized approximation schemes for minimizing the weighted makespan on identical parallel machines. *Journal of Combinatorial Optimization*, 47:34.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Zhang, Y., Wang, X., and Li, Q. (2023). An improved ant colony optimization algorithm for flow shop scheduling problems with sequence-dependent setup times. *International Journal of Production Research*, 61(8):2456–2472.

Zuo, L., Zhang, J., Shi, Z., and Fan, B. (2026). Single machine scheduling with a restricted rate-modifying activity to minimize the weighted makespan. *Discrete Applied Mathematics*, 380:89–100.